

# Završni rad - Streamlit app with PDF

**asp index last3 mi last3 zi** Unable to display visualization

```
This is my app.py: import streamlit as st
import pandas as pd
import fitz
```

```
st.set_page_config(page_title="ASP MVP", layout="wide")
```

```
df = pd.read_csv("data/asp_index_last3_mi_last3_zi.csv")
```

```
st.title("MVP za završni rad")
```

```
goal = st.selectbox("Cilj", ["prolaz (2)", "petica (5)"], key="goal")
days_left = st.slider("Dana do ispita", 0, 14, 1, key="days_left")
session_minutes = st.slider("Vrijeme za study blok (min/dan)", 30, 240, 120, key="session_minutes")
```

```
total_budget = session_minutes * max(1, days_left)
```

```
@st.cache_data
```

```
def render_pdf_page(pdf_path: str, page_1based: int, dpi: int = 150) -> bytes:
```

```
    doc = fitz.open(pdf_path)
    page_index = page_1based - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    pix = page.get_pixmap(dpi=dpi)
    img = pix.tobytes("png")
    doc.close()
    return img
```

```
def get_weights():
```

```
    w_eff = 5.0
    w_points = 2.0
    w_freq = 0.5
```

```
    if goal == "prolaz (2)":
        w_detail = -3.0
        w_diff = -1.5
```

```
    else:
        w_detail = 1.0
        w_diff = -0.5
```

```
    if days_left <= 1:
        w_eff += 2.0
        w_points += 1.0
        w_detail -= 2.0
```

```

return w_eff, w_points, w_detail, w_freq, w_diff

def score_breakdown(row):
    points = float(row["points"])
    time_est = float(row["time_est"])
    eff = points / time_est if time_est > 0 else 0.0
    is_detail = 1.0 if row["type"] == "detail" else 0.0
    freq = float(row.get("frequency_score", 0.0))
    diff = float(row.get("difficulty_guess", 0.0))

    w_eff, w_points, w_detail, w_freq, w_diff = get_weights()

    parts = {
        "efficiency": w_eff * eff,
        "points": w_points * points,
        "detail": w_detail * is_detail,
        "frequency": w_freq * freq,
        "difficulty": w_diff * diff
    }

    total = float(sum(parts.values()))
    return parts, total, {
        "eff": eff,
        "points": points,
        "time_est": time_est,
        "is_detail": is_detail,
        "freq": freq,
        "diff": diff,
        "weights": {
            "w_eff": w_eff,
            "w_points": w_points,
            "w_detail": w_detail,
            "w_freq": w_freq,
            "w_diff": w_diff
        }
    }

def score_task(row) -> float:
    _, total, _ = score_breakdown(row)
    return total

def build_plan(df_in: pd.DataFrame, minutes_budget: int) -> pd.DataFrame:
    df_local = df_in.copy()
    df_local["score"] = df_local.apply(score_task, axis=1)
    df_sorted = df_local.sort_values("score", ascending=False)

    selected_rows = []
    total_time = 0.0

    for _, row in df_sorted.iterrows():
        t = float(row["time_est"])

```

```

    if total_time + t <= float(minutes_budget):
        selected_rows.append(row)
        total_time += t

    return pd.DataFrame(selected_rows)

plan_df = build_plan(df, total_budget)

left, right = st.columns([1, 1])

with left:
    st.subheader("Preporuceni zadaci")

    if plan_df.empty:
        st.warning("Nema zadataka koji stanu u ovaj vremenski budzet. Povecaj minute ili provjeri time_est.")
        st.stop()

    st.caption(f"Ukupni budzet: {int(total_budget)} min (session_minutes * max(1, days_left))")

    total_points = float(plan_df["points"].sum())
    total_time = float(plan_df["time_est"].sum())
    avg_eff = (total_points / total_time) if total_time > 0 else 0.0

    m1, m2, m3 = st.columns(3)
    m1.metric("Ukupno bodova", f"{total_points:.0f}")
    m2.metric("Ukupno minuta", f"{total_time:.0f}")
    m3.metric("Prosjecni efficiency", f"{avg_eff:.3f}")

    st.dataframe(
        plan_df[["exam_type", "exam_date", "task_no", "points", "time_est", "type", "frequency_score", "difficulty_guess", "score"]],
        use_container_width=True
    )

with right:
    st.subheader("Pregled zadatka")

    task_labels = plan_df.apply(
        lambda r: f"{r['exam_type']} {r['exam_date']} - Zadatak {r['task_no']} (pts {r['points']}, {r['time_est']} min)",
        axis=1
    ).tolist()

    if "selected_task_label" not in st.session_state or st.session_state.selected_task_label not in task_labels:
        st.session_state.selected_task_label = task_labels[0]

    selected_task_label = st.selectbox(
        "Odaberi zadatak za pregled:",
        task_labels,

```

```

    key="selected_task_label"
)

idx = task_labels.index(selected_task_label)
selected_task = plan_df.iloc[idx]

pdf_path = f"static/{selected_task['pdf']}"
page = int(selected_task["page"])

st.write(f"PDF: {selected_task['pdf']}, stranica {page}")

# Centered preview by using a narrower middle column
_, mid, _ = st.columns([1, 3, 1])
with mid:
    try:
        img_bytes = render_pdf_page(pdf_path, page)
        st.image(img_bytes, width=700)
    except Exception as e:
        st.error(f"Ne mogu otvoriti PDF stranicu: {e}")

st.subheader("Zasto je odabran ovaj zadatak")

parts, total_score, meta = score_breakdown(selected_task)

for key, value in parts.items():
    if value > 0:
        st.write(f"+ {key}: {value:.2f}")
    elif value < 0:
        st.write(f"- {key}: {value:.2f}")

st.markdown(f"***Ukupni score: {total_score:.2f}***")

with st.expander("Detalji (ulazi i tezine)":
    st.write({
        "inputs": {
            "points": meta["points"],
            "time_est": meta["time_est"],
            "efficiency": meta["eff"],
            "type": "detail" if meta["is_detail"] == 1 else "core",
            "frequency_score": meta["freq"],
            "difficulty_guess": meta["diff"]
        },
        "weights": meta["weights"]
    })

```

Zelim dodati chatbota koji odgovara na pitanja kada korisniku nije jasan zadatak. Treba mu proslijediti tekst zadatka. Tekst zadatka treba se dobiti iz pdf-a. LLM runnam lokalno, Llama 3.1. Korisnik treba samo imati opciju da klikne ne razumijem zadatak nakon cega mu LLM odmah kreće objasnjavati zadatak i rjesenje koje sam mora skuhati. Bitno je radi UX-a da LLM odmah krene sa odgovorom, a ne da se cini kao da LLM ne odgovara. Bitno je da krene pisati odgovor i caractere dio po dio, ne mora odmah izbaciti cijeli chunk teksta kao odgovor. Taj LLM treba biti nalijepljen

desno, dakle sveukupno aplikacija treba imati 3 stupca. Prvi stupac u kojem se mogu namijestati ciljevi, dani pred ispit, itd. Drugi stupac koji samo prikazuje pdf, i treci stupac s chatom koji ima samo opciju ne razumijem, a ne i opciju da ga se prompta nesto drugo. Bez nepotrebnog hardkodiranja. Slobodno me pitaj neka dodatna pitanja.

Evo kako bih to složio, bez nepotrebnog hardkodiranja, s 3 stupca (kontrole, PDF, chat), s automatskim izvlačenjem teksta zadatka iz PDF-a i sa "streaming" ispisom tokena da UX odmah izgleda živo.

Napomena: ti već imaš CSV indeks koji mapira `pdf` i `page` po zadatku, pa je "tekst zadatka" najprirodnije izvući baš iz te stranice PDF-a (PyMuPDF `get_text`). To je konzistentno s time da su zadaci stvarno u tim PDF-ovima.

ZI\_svi\_do\_2026 MI\_svi\_do\_2026

## Ključne ideje implementacije

- **Ekstrakcija teksta zadatka:** `page.get_text("text")` + malo čišćenja.
- **Chat UI:** desni stupac ima samo gumb "Ne razumijem zadatak". Nema `chat_input`.
- **Streaming:** generator koji yield-a komadiće teksta i update-a placeholder (`st.empty()`), tako da se tekst pojavljuje "znak po znak" (ili chunk po chunk).
- **LLM backend bez hardkodiranja:** preko env varijabli biraš backend:
  - `LLM_BACKEND=ollama + OLLAMA_MODEL=llama3.1`
  - ili `LLM_BACKEND=llamacpp + LLAMA_CPP_MODEL_PATH=/putanja/do/model.gguf`

Ispod je kompletan rewrite tvog `app.py` (možeš copy-paste). Zadržao sam tvoju logiku za scoring, dodao srednji stupac za PDF, i treći za chat.

```

import os
import time
import streamlit as st
import pandas as pd
import fitz # PyMuPDF

st.set_page_config(page_title="ASP MVP", layout="wide")

# Ako želiš prebaciti na apsolutnu putanju, stavi npr. u env var:
# INDEX_CSV=/mnt/data/asp_index_last3_mi_last3_zi.csv
INDEX_CSV = os.getenv("INDEX_CSV", "data/asp_index_last3_mi_last3_zi.csv")
STATIC_DIR = os.getenv("STATIC_DIR", "static")

df = pd.read_csv(INDEX_CSV)

st.title('MVP za završni rad')

# -----
# PDF render + text extraction
# -----

@st.cache_data
def render_pdf_page(pdf_path: str, page_lbased: int, dpi: int = 150) -> bytes:
    doc = fitz.open(pdf_path)
    page_index = page_lbased - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    pix = page.get_pixmap(dpi=dpi)
    img = pix.tobytes("png")
    doc.close()
    return img

@st.cache_data
def extract_pdf_page_text(pdf_path: str, page_lbased: int) -> str:
    doc = fitz.open(pdf_path)
    page_index = page_lbased - 1
    if page_index < 0 or page_index >= len(doc):
        doc.close()
        raise ValueError("Stranica ne postoji u PDF-u.")
    page = doc[page_index]
    text = page.get_text("text") or ""
    doc.close()

    # Minimalno čišćenje, bez agresivnog hardkodiranja.
    text = text.replace("\u00a0", " ")
    text = "\n".join([ln.rstrip() for ln in text.splitlines()])
    text = text.strip()
    return text

# -----
# Scoring (tvoje, minimalno dirano)
# -----

goal = st.selectbox("Cilj", ["prolaz (2)", "petica (5)"], key="goal")
days_left = st.slider("Dana do ispita", 0, 14, 1, key="days_left")
session_minutes = st.slider("Vrijeme za study blok (min/dan)", 30, 240, 120, key="session_minutes")

total_budget = session_minutes * max(1, days_left)

def get_weights():
    w_eff = 5.0
    w_points = 2.0
    w_freq = 0.5

    if goal == "prolaz (2)":
        w_detail = -3.0
        w_diff = -1.5
    else:
        w_detail = 1.0
        w_diff = -0.5

    if days_left <= 1:

```

```

        w_eff += 2.0
        w_points += 1.0
        w_detail -= 2.0

    return w_eff, w_points, w_detail, w_freq, w_diff

def score_breakdown(row):
    points = float(row["points"])
    time_est = float(row["time_est"])
    eff = points / time_est if time_est > 0 else 0.0
    is_detail = 1.0 if row["type"] == "detail" else 0.0
    freq = float(row.get("frequency_score", 0.0))
    diff = float(row.get("difficulty_guess", 0.0))

    w_eff, w_points, w_detail, w_freq, w_diff = get_weights()

    parts = {
        "efficiency": w_eff * eff,
        "points": w_points * points,
        "detail": w_detail * is_detail,
        "frequency": w_freq * freq,
        "difficulty": w_diff * diff
    }

    total = float(sum(parts.values()))
    return parts, total, {
        "eff": eff,
        "points": points,
        "time_est": time_est,
        "is_detail": is_detail,
        "freq": freq,
        "diff": diff,
        "weights": {
            "w_eff": w_eff,
            "w_points": w_points,
            "w_detail": w_detail,
            "w_freq": w_freq,
            "w_diff": w_diff
        }
    }

def score_task(row) -> float:
    _, total, _ = score_breakdown(row)
    return total

def build_plan(df_in: pd.DataFrame, minutes_budget: int) -> pd.DataFrame:
    df_local = df_in.copy()
    df_local["score"] = df_local.apply(score_task, axis=1)
    df_sorted = df_local.sort_values("score", ascending=False)

    selected_rows = []
    total_time = 0.0

    for _, row in df_sorted.iterrows():
        t = float(row["time_est"])
        if total_time + t <= float(minutes_budget):
            selected_rows.append(row)
            total_time += t

    return pd.DataFrame(selected_rows)

plan_df = build_plan(df, total_budget)

if plan_df.empty:
    st.warning("Nema zadatka koji stanu u ovaj vremenski budzet. Povecaj minute ili provjeri time_est")
    st.stop()

# -----
# Task picker (shared)
# -----

task_labels = plan_df.apply(
    lambda r: f"{{r['exam_type']}} {{r['exam_date']}} - Zadatak {{r['task_no']}} (pts {{r['points']}}, {{r['time_est']}})",
    axis=1
).tolist()

```

```

if "selected_task_label" not in st.session_state or st.session_state.selected_task_label not in task_labels:
    st.session_state.selected_task_label = task_labels[0]

selected_task_label = st.selectbox(
    "Odaberi zadatak za pregled:",
    task_labels,
    key="selected_task_label"
)

idx = task_labels.index(selected_task_label)
selected_task = plan_df.iloc[idx]

pdf_path = os.path.join(STATIC_DIR, str(selected_task["pdf"]))
page_lbased = int(selected_task["page"])

# -----
# LLM client (ollama ili llama.cpp), bez hardkodiranja
# -----

def build_explain_prompt(task_text: str) -> str:
    # Namjerno fiksni prompt, jer user nema chat input.
    # Fokus: razumijevanje + rješenje + provjere tipičnih zamki.
    return f"""
Ti si asistent za učenje predmeta "Algoritmi i strukture podataka".
Korisnik je kliknuo da ne razumije zadatak. Objasni zadatak i napravi rješenje.

Pravila odgovora:
- Prvo objasni što se traži, u par točaka.
- Zatim izvedi rješenje korak po korak.
- Ako je zadatak programski: daj čisti C++ kod (ili pseudokod ako je prikladnije) i objasni ga.
- Na kraju navedi najčešće pogreske i kratku provjeru točnosti.
- Ne spominji da si model, ne spominji prompt, ne postavlja pitanja korisniku.

TEKST ZADATKA (iz PDF-a):
{task_text}
""".strip()

def stream_from_ollama(prompt: str):
    # pip install ollama
    import ollama
    model = os.getenv("OLLAMA_MODEL", "llama3.1")

    # chat API radi stabilnije za streaming
    stream = ollama.chat(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )
    for chunk in stream:
        # chunk je dict, najčešće chunk["message"]["content"]
        msg = chunk.get("message", {})
        content = msg.get("content", "")
        if content:
            yield content

def stream_from_llamacpp(prompt: str):
    # pip install llama-cpp-python
    from llama_cpp import Llama

    model_path = os.getenv("LLAMA_CPP_MODEL_PATH", "")
    if not model_path:
        raise RuntimeError("Nedostaje LLAMA_CPP_MODEL_PATH env var (putanja do .gguf modela).")

    n_ctx = int(os.getenv("LLAMA_CPP_N_CTX", "4096"))
    n_threads = int(os.getenv("LLAMA_CPP_THREADS", str(os.cpu_count() or 4)))

    llm = Llama(
        model_path=model_path,
        n_ctx=n_ctx,
        n_threads=n_threads,
        # po potrebi: n_gpu_layers=int(os.getenv("LLAMA_CPP_GPU_LAYERS", "0"))
    )

    # create completion streaming
    from llama_cpp import Llama
    completion = llm.create_completion(
        prompt=prompt,
        stream=True,
        n_ctx=n_ctx,
        n_threads=n_threads,
        # po potrebi: n_gpu_layers=int(os.getenv("LLAMA_CPP_GPU_LAYERS", "0"))
    )
    for chunk in completion:
        yield chunk["content"]

```

```

for out in llm.create_completion(
    prompt=prompt,
    max_tokens=int(os.getenv("LLAMA_MAX_TOKENS", "900")),
    temperature=float(os.getenv("LLAMA_TEMPERATURE", "0.2")),
    stream=True,
):
    choice = (out.get("choices") or [{}])[0]
    text = choice.get("text", "")
    if text:
        yield text

def stream_llm_answer(task_text: str):
    backend = os.getenv("LLM_BACKEND", "ollama").strip().lower()
    prompt = build_explain_prompt(task_text)

    if backend == "ollama":
        yield from stream_from_ollama(prompt)
    elif backend == "llamacpp":
        yield from stream_from_llamacpp(prompt)
    else:
        raise RuntimeError("Nepoznat LLM_BACKEND. Koristi 'ollama' ili 'llamacpp'.")

# -----
# 3-column layout
# -----

col1, col2, col3 = st.columns([1, 2, 1])

with col1:
    st.subheader("Preporuceni zadaci")
    st.caption(f"Ukupni budzet: {int(total_budget)} min (session_minutes * max(1, days_left))")

    total_points = float(plan_df["points"].sum())
    total_time = float(plan_df["time_est"].sum())
    avg_eff = (total_points / total_time) if total_time > 0 else 0.0

    m1, m2, m3 = st.columns(3)
    m1.metric("Ukupno bodova", f"{total_points:.0f}")
    m2.metric("Ukupno minuta", f"{total_time:.0f}")
    m3.metric("Prosjecni efficiency", f"{avg_eff:.3f}")

    st.dataframe(
        plan_df[["exam_type", "exam_date", "task_no", "points", "time_est", "type", "frequency_score",
        use_container_width=True
    )

with col2:
    st.subheader("PDF pregled")
    st.write(f"PDF: {selected_task['pdf']}, stranica {page_lbased}")

    # Centered preview
    _, mid, _ = st.columns([1, 3, 1])
    with mid:
        try:
            img_bytes = render_pdf_page(pdf_path, page_lbased)
            st.image(img_bytes, width=850)
        except Exception as e:
            st.error(f"Ne mogu otvoriti PDF stranicu: {e}")

    st.subheader("Zasto je odabran ovaj zadatak")
    parts, total_score, meta = score_breakdown(selected_task)

    for key, value in parts.items():
        if value > 0:
            st.write(f"+ {key}: {value:.2f}")
        elif value < 0:
            st.write(f"- {key}: {value:.2f}")

    st.markdown(f"***Ukupni score: {total_score:.2f}***")

    with st.expander("Detalji (ulazi i tezine)"):
        st.write({
            "inputs": {
                "points": meta["points"],
                "time_est": meta["time_est"],
                "efficiency": meta["eff"]
            }
        })

```

```

        efficiency: meta["eff"],
        "type": "detail" if meta["is_detail"] == 1 else "core",
        "frequency_score": meta["freq"],
        "difficulty_guess": meta["diff"]
    },
    "weights": meta["weights"]
})

with col3:
    st.subheader("Pomoc (LLM)")

    if "chat_messages" not in st.session_state:
        st.session_state.chat_messages = []
        st.session_state.chat_messages.append({
            "role": "assistant",
            "content": "Klikni 'Ne razumijem zadatak' i objasniti cu zadatak i rjesenje."
        })

    # Render chat history
    for m in st.session_state.chat_messages:
        with st.chat_message(m["role"]):
            st.markdown(m["content"])

    st.caption("Ovdje nema rucnog unosa. Samo instant objasnjenje za trenutno odabrani zadatak.")

    clicked = st.button("Ne razumijem zadatak", use_container_width=True)

    if clicked:
        # Extract task text from PDF page
        try:
            task_text = extract_pdf_page_text(pdf_path, page_lbased)
        except Exception as e:
            st.error(f"Ne mogu izvuci tekst zadatka iz PDF-a: {e}")
            st.stop()

        # Optional: ako je tekst prazan, daj fallback poruku
        if not task_text.strip():
            st.error("Izvucenti tekst je prazan. Moguce je da je stranica skenirana slika bez teksta.")
            st.stop()

        # Add a synthetic user event to chat for clarity
        st.session_state.chat_messages.append({
            "role": "user",
            "content": "Ne razumijem zadatak. Objasni mi ga i rijesi."
        })

        with st.chat_message("user"):
            st.markdown("Ne razumijem zadatak. Objasni mi ga i rijesi.")

        # Stream assistant answer
        with st.chat_message("assistant"):
            placeholder = st.empty()
            acc = ""

            # Brzi "kick" da se odmah nesto prikaze, i prije prvog tokena
            placeholder.markdown('Krecem s objasnjenjem...')

            try:
                for chunk in stream_llm_answer(task_text):
                    acc += chunk
                    placeholder.markdown(acc)
                    # Malo uspori ako zelis "typing" osjecaj i kad LLM salje vece chunkove
                    # time.sleep(0.01)
            except Exception as e:
                placeholder.markdown("")
                st.error(f"LLM greska: {e}")
                st.stop()

        st.session_state.chat_messages.append({
            "role": "assistant",
            "content": acc
        })

```

## Bitne napomene (da te ne ubije kasnije)

1. Ako su ti PDF-ovi skenirani kao slika, `get_text()` će vratiti prazno. U tom slučaju imaš 2 opcije:

- (bolje) unaprijed generirati tekstualni "dataset" za svaku stranicu (offline OCR), i u appu samo dohvatiti tekst.
- (brže) ubaciti OCR fallback za samo tu stranicu, ali to je sporije i UX pati.

2. Streaming backendovi:

- Ollama streaming radi super i jednostavan je za local.
- llama-cpp-python streaming radi ok, ali init modela je skup, pa ćeš htjeti cache-ati instancu modela (npr. `@st.cache_resource` za `Llama(...)`) kad kreneš peglati performanse.

Ako hoćeš, u sljedećoj poruci ti mogu poslati verziju gdje je `Llama(...)` cache-an kao resource (da se ne učitava svaki klik), i gdje chat automatski resetira odgovor kad promijeniš zadatak (da korisnik ne gleda stari odgovor za novi PDF).

Sources

koja je razlika između RAG, prompt injectiona i fine-tunanja?

Odlično pitanje. Ovo je **ključna razlika u arhitekturi LLM sustava**, pogotovo za tvoj projekt gdje objašnjavaš zadatke iz PDF-ova.

Objasnit ću ti precizno, bez marketing magle.

## 1 Prompt injection (u pozitivnom smislu: ubacivanje konteksta u prompt)

U praksi znači:

U prompt ručno ubaciš dodatni tekst koji model treba znati.

Primjer kod tebe:

Objasni zadatak.

TEKST ZADATAKA:  
[tekst iz PDF-a]

To je to. Nema baze, nema treniranja, nema memorije.

## Što se događa tehnički?

Model dobije sve kao jedan veliki input kontekst i generira odgovor.

## Prednosti

- Najjednostavnije
- Nema dodatne infrastrukture

- Savršeno za MVP

## Nedostaci

- Kontekst je ograničen veličinom prozora (npr. 4k, 8k, 32k tokena)
- Model nema “stvarno znanje” o tvom predmetu, samo reagira na dani tekst
- Ako ne ubaciš nešto u prompt, model to “ne zna”

## 2 RAG (Retrieval-Augmented Generation)

RAG znači:

Prvo dohvatim relevantne informacije iz baze, pa ih onda ubacim u prompt.

Dakle:

1. Korisnik klikne
2. Sustav pretraži bazu (npr. embedding search po svim ispitima)
3. Izvuče relevantne dijelove
4. Ubaci ih u prompt
5. Model generira odgovor

## Arhitektura

```

User → Retriever (embedding search) → top-k tekstovi
      ↓
    Prompt + retrieved context
      ↓
      LLM
  
```

## Razlika od običnog prompt injectiona

Kod prompt injectiona TI odlučuješ što ide u prompt.

Kod RAG-a model dobije **automatski pronađen kontekst iz baze znanja**.

## Prednosti

- Možeš imati tisuće stranica materijala
- Model “zna” sve iz kolegija bez fine-tuninga
- Skalabilno

## Nedostaci

- Potrebna je embedding baza
- Malo kompleksnija infrastruktura
- Moraš paziti na kvalitetu retrievala

## 3 Fine-tuning

Fine-tuning znači:

Mijenjaš težine modela da se trajno prilagodi tvom zadatku.

To NIJE dodavanje konteksta.

To je:

```
Uzmeš Llama 3.1
↓
Dodaš dataset (npr. 10 000 parova: zadatak → rješenje)
↓
Treniraš model
↓
Dobiješ novi model koji "prirodno" rješava ASP zadatke
```

Model tada "internalizira" znanje.

---

### Prednosti

- Model dobije stil, strukturu i ponašanje
- Manje treba prompt inženjeringa
- Brži inference jer ne trebaš ubacivati puno konteksta

### Nedostaci

- Skupo
  - Treba kvalitetan dataset
  - Teško za održavanje
  - Ne zna nove podatke ako ih ne retreniraš
- 

## Najvažnija razlika (jedna rečenica)

- **Prompt injection** = ubaciš tekst u prompt.
  - **RAG** = automatski pronađeš tekst pa ga ubaciš u prompt.
  - **Fine-tuning** = promijeniš mozak modela.
- 

## Važno: "Prompt injection" kao sigurnosni pojam

U sigurnosnom kontekstu "prompt injection" znači nešto drugo:

Zlonamjerni tekst pokušava preuzeti kontrolu nad modelom.

To je druga tema i nema veze s tvojom upotrebom.

---

## ⚡ Što je najbolje za tvoj ASP projekt?

Za tebe:

### Trenutno (MVP)

Prompt injection je savršen.

Jer:

- Imaš točno određeni zadatak
- Ubacuješ tekst stranice
- Model generira objašnjenje

To je idealno.

### Kada prijeći na RAG?

Kad:

- Imaš 200+ ispita
- Želiš da model povezuje koncepte
- Želiš da model zna cijelu skriptu

### Fine-tuning?

Realno:

- Overkill za sada
- Nemaš dovoljno strukturiranih parova
- RAG će ti dati 80% rezultata uz 20% truda

## 🔥 Brza tablica

| Feature                | Prompt injection | RAG   | Fine-tuning |
|------------------------|------------------|-------|-------------|
| Jednostavnost          | ★★★★★            | ★★★   | ★           |
| Skalabilnost           | ★                | ★★★★★ | ★★★★        |
| Trajna promjena modela | ✗                | ✗     | ✓           |
| Treba dataset          | ✗                | ✗     | ✓           |
| Idealno za MVP         | ✓                | 😬     | ✗           |

Ako želiš, mogu ti sada:

- objasniti kako bi tvoj trenutni app evoluirao u RAG verziju
- ili ti nacrtati arhitekturu profesionalne verzije tvoje aplikacije

Što želiš sljedeće?

ako hocu da moj LLM moze predavati o tom podrucju koji je to nacin?

Ako želiš da tvoj LLM “može predavati ASP” na način da daje jasna objašnjenja, rješava zadatke u tvom stilu i poziva se na tvoje materijale, onda je najbolji put gotovo uvijek kombinacija:

### **RAG za znanje + fine-tuning za stil i ponašanje.**

A ako moraš izabrati samo jedan, onda:

- **RAG** ako želiš da zna sadržaj iz skripti, ispita i prezentacija, i da se to lako ažurira.
- **Fine-tuning** ako želiš da ima tvoj “profesorski” stil objašnjavanja i standardiziranu strukturu rješenja.

Evo kako to izgleda praktično, ovisno što ti znači “predavati”.

## **1) Želiš da zna tvoje materijale i uvijek bude točan prema njima**

**RAG** je prvi izbor.

Zašto: “predavanje” o području zahtijeva puno činjenica i detalja. Fine-tuning nije dobar “storage” za znanje, nego za ponašanje. RAG ti daje direktan pristup izvoru.

Kako bi to radio:

- Uvezeš sve PDF-ove (skripta, prezentacije, ispiti).
- Razbiješ ih na male blokove (npr. 300–800 tokena).
- Napraviš embeddinge i spremiš u vektorsku bazu.
- Kad korisnik pita ili klikne “Ne razumijem”, sustav povuče top-k relevantnih blokova i ubaci u prompt.

To znači da LLM “predaje” koristeći tvoje izvore kao otvorenu knjigu.

## **2) Želiš da predaje u tvom stilu i “uvijek” daje strukturu kao na faksu**

Tu je **fine-tuning** najbolji.

Fine-tuning ti daje:

- konzistentan format: "što se traži → ideja → koraci → rješenje → provjera"
- način objašnjavanja (način na koji ti želiš da "profesor" zvuči)
- manje promptovanja

Ali: bez RAG-a, fine-tuned model i dalje može halucinirati, i neće automatski znati najnovije ili specifične detalje iz baš tog PDF-a.

---

### 3) Želiš "pravog predavača" koji:

- zna cijeli predmet,
  - rješava zadatke,
  - referencira gradivo,
  - i zvuči kao jedan isti "profesor"
- Najbolje rješenje je:

#### RAG + fine-tuning (ili barem "instruction tuning")

- RAG daje činjenice i kontekst iz materijala
- fine-tuning daje pedagoški stil, strukturu i "policy" ponašanja

Ovo je industrijski standard za edukacijske asistente.

---

### Najkraće pravilo

Ako ti je cilj "predavati gradivo", prvo napravi RAG.

Ako ti je cilj "predavati kao ti", dodaj fine-tuning.

---

### Što bih ja napravio za tvoj slučaj (ASP zadaci)

1. Sad: izvuci tekst zadatka iz PDF-a i ubaci u prompt (što već radiš ili planiraš).
  2. Kad to radi: dodaj RAG preko svih ispita i prezentacija, da može objašnjavati i koncepte, ne samo taj zadatak.
  3. Tek kad imaš dovoljno primjera rješenja, dodaš fine-tuning da se ponaša kao "ASP instruktor".
- 

### Kako dobiješ dataset za fine-tuning u praksi

Najbolji dataset nije "teorija", nego parovi tipa:

- **Input:** tekst zadatka + eventualno relevantni izvod iz skripte
- **Output:** tvoje rješenje i objašnjenje, u standardnoj strukturi

Minimum koji ima smisla: recimo 200–500 kvalitetnih primjera.  
Bolje: 1k+.

---

## Jedna bitna zamka

“Predavati o području” često ljudi misle kao “da model zna sve i da je uvijek u pravu”.  
To bez izvora (RAG) vrlo brzo postane problem, jer model može zvučati uvjerljivo i kad nije točan.  
Zato: za predmet i zadatke, RAG je temelj.

---

Ako mi kažeš jednu stvar, mogu ti točno preporučiti smjer:

Želiš li da model primarno bude:

1. “rješavač zadataka s objašnjenjem”, ili
2. “tutor koji predaje teoriju i konceptualno”, ili
3. oboje?